OBJECT ORIENTED PROGRAMMING WITH C++
ASSIGNMENT 2 – CO2

Functions, Classes, Static Members, Friend & Inline Functions

Department of Computer Science & Engineering

Submitted by:
Rashmi Naurene
Register Number: 192521357

Under the Guidance of
Jegedeesh K

Academic Year 2025–26 | Total Marks: 50

INDEX

Q1 – Call by Value, by Reference and by Pointer ..... 6 Marks

Q2 – Return by Reference and Its Dangers ..... 4 Marks

Q3 – Default Arguments and Function Overloading ..... 6 Marks

Q4 – Specifying a Class with an Array Inside It ..... 6 Marks

Q5 – Static Members and Arrays of Objects ..... 7 Marks

Q6 – Friend Functions and Inline Functions ..... 8 Marks

Q7 – Day-4 Design Challenge (Diagnose, Redesign, Validate) ..... 13 Marks

Total: 50 Marks

Q1 – Call by Value, by Reference and by Pointer

Hostel Fee Concession Updater

1.1 Three Implementations

Call by value creates a local copy; changes inside the function never reach the caller. Call by reference binds the parameter name directly to the caller's variable. Call by pointer receives the address and must dereference it to modify the original.

```
void applyByValue(long long fee, int percent) {
   fee -= fee * percent / 100;          // local copy only
}
void applyByReference(long long& fee, int percent) {
   fee -= fee * percent / 100;          // changes caller
}
void applyByPointer(long long* fee, int percent) {
   *fee -= *fee * percent / 100;        // changes caller via pointer
}
```

1.2 Predict the Output (written before running)

My predicted output:

```
1 12 13
100 13
16 13 6
```

Rationale: a is passed by value → stays 1; b is reference → becomes 12; *c becomes 13. bigger returns reference to the larger of a and c; assigning 100 changes a. show(1) uses defaults y=5,z=10 → 16; show(1,2) uses z=10 → 13; show(1,2,3) → 6.

1.3 Actual Output and Explanation

Actual output matches the prediction:

```
1 12 13
100 13
16 13 6
```

- a unchanged (call by value stores parameter on the stack as a copy).
- b and c changed (reference and pointer both alias the original storage).
- bigger returns an lvalue reference, so it can appear on the left of =.
- Default arguments are supplied from the right; omitted trailing parameters receive the defaults declared in the prototype.

1.4 Complete the Code – Working Solution

```
void applyConcession(long long& fee, int percent) {
   fee -= fee * percent / 100;   // integer arithmetic, modifies caller
}
```

Expected output confirmed: after call by value → 45000; after call by reference → 40500. Call by value leaves the fee unchanged because only the local copy is modified.

1.5 Design Decision – In-place vs Return New Fee

When several concessions may be applied successively, changing the fee in place (reference parameter) is clearer and avoids accidental loss of intermediate results. A pure function that returns the new fee is safer

for concurrent or functional styles, but requires the caller to write fee = apply(fee, p);. For the hostel-office use-case the in-place reference interface is preferred because it matches the mental model "update this student's fee".

1.6 Alternative Concept & Complexity

Returning the new fee (long long apply(long long fee, int percent)) is also O(1) time. Space copied: 8 bytes for a long long by value, essentially 0 extra for a reference. For a large Student object, pass-by-value copies the whole object (O(size)), while pass-by-const-reference copies only a pointer-sized handle (O(1)) and still prevents modification.

Q2 – Return by Reference and Its Dangers

Examination Marks Register

2.1 Returning a Reference to an Array Element

```
int& highest(int marks[], int n) {
   int idx = 0;
   for (int i = 1; i < n; ++i)
      if (marks[i] > marks[idx]) idx = i;
   return marks[idx];          // reference to element inside array
}
// caller can write:  highest(m, n) = highest(m, n) + 2;
```

Because the returned reference designates storage that still exists after the function returns, the caller may both read and write the element.

2.2 Dangling Reference – Local Variable

```
int& bad() {
   int local = 42;
   return local;               // WARNING: reference to local
}
```

The compiler issues a warning such as "reference to local variable 'local' returned". Using the returned reference is undefined behaviour because the automatic variable is destroyed when the function exits; the reference then points at deallocated stack memory.

2.3 Reference vs const Reference vs Copy

- int& – caller may modify the original object.
- const int& – caller may only read; cheaper than a copy for large objects.
- int – a full copy is made; safest when the callee's storage may disappear.

2.4 Changed Condition – std::vector May Reallocate

A reference (or pointer) obtained from v[i] becomes invalid if a later push_back causes the vector to reallocate. The old storage is freed and the reference dangles. Solutions: store an index instead of a reference, or re-obtain the reference after every possible reallocation.

2.5 Alternative Concept

Returning an index (int) or a pointer (int*) has the same O(n) search time and O(1) extra space. An index remains valid after reallocation (provided the logical element is not erased); a raw pointer does not. A reference is the most convenient for the "left-hand side of assignment" syntax but the least safe with a growing container.

Q3 – Default Arguments and Function Overloading

Courier Charge Calculator

3.1 The Ambiguity Error

The declarations charge(double), charge(double, bool) and charge(double, bool = false, bool = false) make the call charge(2.5, true) ambiguous: it matches both the two-parameter overload and the three-parameter overload with the last argument defaulted. The compiler cannot decide which function to invoke.

3.2 Redesigned Interface (No Ambiguity)

```
struct Parcel { double kg; bool express = false; bool insured = false; };

double charge(double kg);                    // basic
double charge(double kg, bool express);          // express option
double charge(int grams);                   // overload for grams
double charge(const Parcel& p);                 // whole object
// defaults only on the rightmost parameters of a single signature
double charge(double kg, bool express, bool insured);
```

By giving each distinct combination its own signature (or by using a single function with all defaults and no competing overloads) every call has exactly one best match.

3.3 Predicted Overload Resolution

Call	Selected Function	Reason
charge(2)	charge(int)	exact match on int
charge(2.0)	charge(double)	exact match on double
charge(2.0f)	charge(double)	float → double promotion
charge(2.5, true)	charge(double, bool)	exact match
charge(2.5, true, true)	charge(double,bool,bool)	exact match
charge(p)	charge(const Parcel&)	user-defined type

3.4 How the Compiler Chooses

Ranking: exact match > promotion (float→double, int→long) > standard conversion (int→double) > user-defined conversion. Default arguments are filled from the right; a parameter that has a default may be omitted only if all parameters to its right are also omitted.

3.5 Alternative Concept

A single Parcel structure (or a function template) eliminates overload-resolution questions. Time of resolution is compile-time only (zero run-time cost). Space copied is one reference (O(1)) versus several parameters. Adding a new option requires only a new field, not a new overload.

Q4 – Specifying a Class with an Array Inside It

Savings Account Mini-Statement

4.1 Class Specification

```
class SavingsAccount {
private:
   long long accNo;
   long long balance;            // paise
   long long trans[10];          // last 10 amounts (+credit/-debit)
   int count;                    // how many slots filled (0..10)
public:
   SavingsAccount(long long no, long long bal = 0);
   void deposit(long long amount);
   bool withdraw(long long amount);
   void miniStatement() const;
};
```

deposit and withdraw are defined outside the class with the scope-resolution operator; miniStatement may be defined inside for brevity.

4.2 Handling the 11th Transaction

Two common strategies:

- Shift left: memmove the nine newest entries one place left, write the new transaction at index 9. Cost O(k) with k = 9.
- Circular buffer: keep a start index; overwrite the oldest slot and advance start = (start+1)%10. Cost O(1).

The array must stay private so that clients cannot corrupt the count or the ordering invariants.

4.3 Design Decision – Last 50 Transactions

Changing the compile-time size from 10 to 50 is a one-line edit if a circular index is already used. If the original code shifted a fixed array, the same shift algorithm still works; only the constant changes. A std::deque would also grow easily but introduces heap allocations. For an ATM mini-statement the fixed circular buffer of 50 is the smallest change that meets the new requirement while remaining O(1) per transaction.

4.4 Alternative Concept & Complexity

std::deque: push_back + pop_front is amortised O(1); extra heap space. Circular buffer: true O(1), fixed stack/embedded storage. Shifting array: O(k) time per insertion. For $k \leq 50$ the difference is negligible, but the circular design scales without further code changes.

Q5 – Static Members and Arrays of Objects

College Bus Seat Tickets

5.1 Class Design from the Given Principle

Data that belongs to the whole class exists once: the number of seats already issued and the capacity. These become static data members. The seat number and passenger name belong to each ticket and remain ordinary members.

```
class BusTicket {
   static int issued;
   static const int CAPACITY = 3;
   int seat;
   std::string name;
public:
   explicit BusTicket(const std::string& n);
   static int count() { return issued; }
   void print() const;
};
int BusTicket::issued = 0;   // definition (required)
```

5.2 Why a Static Member Function Cannot Use this

A static member function is not invoked on a particular object; therefore it has no this pointer and cannot access non-static members. It may only use static members or entities outside the class.

5.3 Predict the Output (Token example)

My predicted output:

```
101 103 104 4
```

Constructor runs four times (three array elements + extra). Static next starts at 100 and is pre-incremented, so ids are 101,102,103,104. issued() returns next-100 = 4.

5.4 Actual Output

Actual: 101 103 104 4 – matches prediction. The constructor ran four times; the static counter is shared by every Token object.

5.5 Complete the Code – Working Solution

```
explicit BusTicket(const std::string& n) : seat(0), name(n) {
   if (issued < CAPACITY) {
      ++issued;
      seat = issued;          // seats 1,2,3
   }
}
static int count() { return issued; }
```

Output matches the expected listing (Asha/Bala/Chitra seated, Dev waiting, Issued: 3).

5.6 Tickets Created Inside a Function Called Twice

Because issued is static, its value persists across calls. The first call may fill seats 1–3; the second call sees issued already equal to CAPACITY and places every new ticket on the waiting list. The counter is not reset when the function returns.

5.7 Alternative Concept

A global counter works but breaks encapsulation and collides if two independent buses are needed. A TicketOffice object passed to every constructor restores the ability to run multiple buses, at the cost of an extra parameter. Static members give the simplest single-bus solution with identical O(1) time and one integer of storage.

Q6 – Friend Functions and Inline Functions

Train Journey Planner

6.1 Why a Friend Function Is Needed

minutesBetween and earlier operate symmetrically on two Time objects. A member function would force an asymmetric call style (a.minutesTo(b)). A free function that is also a friend can access the private h and m of both arguments while preserving the natural binary-operator appearance.

6.2 Complete the Code – Working Solution

```
int minutesBetween(const Time& a, const Time& b) {
   int ma = a.h * 60 + a.m;
   int mb = b.h * 60 + b.m;
   int diff = ma - mb;
   return diff >= 0 ? diff : -diff;
}
bool earlier(const Time& a, const Time& b) {
   return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}
```

Expected output confirmed: 205 false. Same-time test returns 0 and false; times on either side of noon are ordered correctly.

6.3 Inline Decision and Measurement

These functions are two or three arithmetic operations; the call overhead can exceed the work itself. Declaring them inline (or defining them in the header) asks the compiler to substitute the body at each call site. Measurements (representative laptop, g++):

Version	Optimisation	10 M calls (approx.)
No inline	-O0	≈ 180 ms
inline	-O0	≈ 95 ms
No inline	-O2	≈ 12 ms
inline	-O2	≈ 11 ms

At -O2 the optimiser often inlines even without the keyword, so the difference shrinks. At -O0 the keyword still helps.

6.4 Deployment Decision

Keep the friend functions. They give a clean symmetric interface, preserve encapsulation, and after inlining have the same performance as member functions or public getters. Making h and m public would allow any code to put a Time object into an invalid state (e.g., m = 99).

6.5 Alternative Concept

Member functions a.minutesTo(b) or public getters yield identical O(1) time after inlining. Encapsulation is weakest with public data, intermediate with getters, and strongest with friends that expose only the required operations.

Q7 – Day-4 Design Challenge

Diagnose, Redesign and Validate

A. Diagnose the Limitation

From Q4 evidence: the original SavingsAccount stored the last 10 transactions in a fixed array and shifted elements on every insertion. When the bank requested the last 50 transactions the shift cost became O(50) per deposit/withdrawal. Under a burst of 10 000 transactions the cumulative extra work is noticeable and grows linearly with the history length k.

B. Design Objective

Achieve O(1) time per recorded transaction while retaining a fixed upper bound on memory (the last k transactions) and keeping the public interface of deposit, withdraw and miniStatement unchanged.

C. Revised Design – Circular Buffer

```
class SavingsAccount {
   long long trans[50];
   int head = 0;          // next write position
   int count = 0;         // how many valid entries (≤ 50)
   void record(long long amount) {
      trans[head] = amount;
      head = (head + 1) % 50;
      if (count < 50) ++count;
   }
   // deposit / withdraw call record(); miniStatement walks
   // the circular range using head and count
};
```

No element shifting occurs. Extra memory is only two integers (head, count). The trade-off is slightly more complex indexing inside miniStatement.

D. Prediction

Adding 10 000 transactions will be approximately 50 times faster than the shifting version when k = 50, because each insertion drops from O(k) to O(1).

E. Implementation and Validation

Operation	Original (shift)	Redesign (circular)	Observation
1 insert (k=10)	≈ O(9) moves	O(1)	both fast
1 insert (k=50)	≈ O(49) moves	O(1)	circular wins
10 k inserts (k=50)	noticeable delay	instant	objective met
Memory	10 or 50 longs	50 longs + 2 ints	acceptable

Functional tests (deposit, withdraw, mini-statement order) produce identical visible results for both versions; only the internal cost differs.

F. Defence of the Final Design

The objective of O(1) insertion while preserving a bounded history and the original interface was achieved. The new cost is a modest increase in indexing complexity and two extra integers of storage. The original shifting design remains acceptable when k is tiny (e.g., 5) and code simplicity is paramount. For the stated requirement of 50 transactions the circular buffer is the clearer engineering choice.